2025 IEEE 68th International Midwest Symposium on Circuits and Systems (MWSCAS)
August 10-13, 2025. Lansing, Michigan, USA

Area-Time Efficient Hardware Design for CRYSTALS-Dilithium

Hien Nguyen* and Tuy Tan Nguyen†
*School of Informatics, Computing, and Cyber Systems, Northern Arizona University, Flagstaff, AZ 86011, USA
†Department of Electrical and Computer Engineering, Florida State University, Tallahassee, FL 32310, USA
tn598@nau.edu, tuy.nguyen@fsu.edu

2025 IEEE 68th International Midwest Symposium on Circuits and Systems (MWSCAS) | 979-8-3315-8934-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/MWSCAS53549.2025.11244431

***Abstract*—CRYSTALS-Dilithium (Dilithium), a digital signature scheme based on lattice cryptography, has recently been designated as a standard for post-quantum digital signatures by the National Institute of Standards and Technology. Known for its strong security guarantees and efficient use of polynomial arithmetic, Dilithium offers a promising foundation for future-proof digital signatures. However, implementing such lattice-based schemes in hardware introduces significant challenges due to the high computational demands and structural complexity of their core operations. To overcome these issues, this work focuses on optimizing Dilithium for field-programmable gate arrays (FPGAs), which offer substantial potential for performance gains through parallelism and customized architectural design. We introduce a novel hardware design that improves performance and area by employing a 2×2 butterfly number-theoretic transform capable of computing two transform stages per cycle, along with a conflict-free memory subsystem that leverages pipelined coefficient rearrangement and an address resolution mechanism to ensure continuous data access. Evaluation on an Xilinx Versal Premium FPGA shows that our design enhanced resource utilization and performance, reaching 328 MHz and an area-time product of 5.4 KLUTs × ms at security level 2.**

***Index Terms*—CRYSTALS-Dilithium, number theoretic transform, FPGA, hardware implementation.**

I. Introduction

As the era of quantum computing approaches, traditional cryptographic standards face unprecedented threats [1], [2]. Among the emerging solutions, CRYSTALS-Dilithium (Dilithium) [3], a lattice-based digital signature scheme, has recently been selected by the National Institute of Standards and Technology (NIST) as a standard for post-quantum digital signatures [4]. Based on structured lattices and designed to support efficient arithmetic over polynomial rings, Dilithium is highly suitable for hardware implementation. However, deploying it efficiently on field-programmable gate array (FPGA) platforms remains a challenge due to the computational intensity of operations such as the number-theoretic transform (NTT) and rejection sampling, all of which require careful architectural design to meet performance and area constraints.

Several recent studies have addressed this problem from different perspectives. In [5] and [6], the authors proposed pipelined or radix-4 NTT architectures with various memory optimizations; however, they compute only one NTT stage per cycle, thereby underutilizing the pipeline. The authors in [7], [8] focused on lightweight and compact implementations targeting low-end FPGAs or application-specific integrated circuits (ASICs), often trading off performance to minimize resource consumption. In [9], the authors proposed unified cryptoprocessors that support multiple post-quantum cryptography (PQC) schemes, including Dilithium and Saber, but are not optimized for the specific demands of Dilithium polynomial arithmetic. Meanwhile, the authors in [10] proposed a sparse polynomial multiplication method to reduce the number of operations, achieving high throughput but relying on structured sparsity and precomputed constants, which limits its generality and flexibility. This paper presents a high-performance FPGA implementation of Dilithium, designed to address the core challenges of hardware-based post-quantum digital signature schemes. The architecture is fully compliant with the Dilithium specification and supports all three NIST-defined security levels (2, 3, and 5), with runtime configurability that allows the system to adapt dynamically to varying performance and security requirements, without the need for hardware modification. To overcome performance bottlenecks observed in prior implementations, our design introduces two key architectural innovations. The first is a deeply pipelined 2×2 butterfly NTT engine capable of executing two transform stages per clock cycle, significantly reducing latency and improving throughput. The second is a conflict-free memory access mechanism that integrates pipelined coefficient flow, an address resolver, and a bit-reversed memory layout, enabling continuous data streaming and minimizing block random access memory (BRAM) access stalls.

The remaining sections of the paper are structured as follows: Section II provides detailed background on NTT and Dilithium. Section III presents our FPGA design methodology, highlighting the key architectural features and optimizations employed. In Section IV, we discuss the experimental results, demonstrating the performance improvements achieved. Finally, Section V concludes the paper with a summary of our findings and potential directions for future research.

II. Background

A. Number theoretic transform

The number theoretic transform (NTT) is a variant of the discrete Fourier transform (DFT) adapted for operations

This work was supported in part by the Department of Electrical and Computer Engineering, FAMU-FSU College of Engineering, and the Center for Advanced Power Systems, Florida State University; and in part by the School of Informatics, Computing, and Cyber Systems, Northern Arizona University. *(Corresponding author: Tuy Tan Nguyen.)*

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on May 06,2026 at 19:12:28 UTC from IEEE Xplore. Restrictions apply.

over finite fields, making it particularly suitable for cryptographic schemes that operate on modular integer arithmetic [2]. It enables efficient polynomial multiplication in the ring $\mathbb{Z}_q[x]/(x^N+1)$, where q is a prime modulus such that a primitive N-th root of unity exists in $\mathbb{Z}_q$.

Given a polynomial $a(x) = \sum_{j=0}^{N-1} a_j x^j$, the NTT maps it to a coefficient vector $A = \text{NTT}(a)$ in the transformed domain:

$$A_k = \sum_{j=0}^{N-1} a_j \cdot \omega^{jk} \mod q, \quad \text{for } k = 0, ..., N-1 \tag{1}$$

where ω is a primitive N-th root of unity modulo q.

The inverse NTT (INTT) restores the original polynomial:

$$a_j = N^{-1} \cdot \sum_{k=0}^{N-1} A_k \cdot \omega^{-jk} \mod q, \quad \text{for } j = 0, ..., N-1 \tag{2}$$

where N^{-1} is the multiplicative inverse of N modulo q.

Polynomial multiplication in Dilithium is performed using the NTT as:

$$c(x) = a(x) \cdot b(x) \mod (x^N + 1, q) \tag{3}$$

Which is efficiently computed by transforming the inputs into the NTT domain:

$$C = \text{INTT}(\text{NTT}(a) \odot \text{NTT}(b)) \tag{4}$$

This approach reduces the computational complexity from $O(N^2)$ to $O(N \log N)$, making it a fundamental operation in high-performance lattice-based cryptographic hardware.

B. Overview of Dilithium

Dilithium is a lattice-based digital signature scheme that achieves post-quantum security by relying on the hardness of the module learning with errors and module short integer solution problems [3]. All operations are defined over the polynomial ring $\mathbb{Z}_q[x]/(x^N+1)$, where $N = 256$ and q is a carefully chosen prime to support efficient NTT implementation.

The key generation procedure begins by sampling a uniformly random matrix $A \in \mathbb{Z}_q^{k \times l}[x]$ from a public seed. Two secret vectors $s_1 \in \mathbb{Z}^l$ and $s_2 \in \mathbb{Z}^k$ are sampled from a centered binomial distribution χ_η. The public key is computed as $t = A \cdot s_1 + s_2 \mod q$, where all polynomial multiplications are performed efficiently using NTT.

To sign a message μ, a short vector $y \leftarrow \chi_{\gamma_1}$ is sampled, and the commitment w is computed as:

$$w = \text{highbits}(A \cdot y \mod q). \tag{5}$$

A hash function is used to generate the challenge polynomial $c = H(w, \mu)$. The response vector is calculated as $z = y + c \cdot s_1$. If z exceeds a predefined norm bound or fails other constraints, the signature is rejected and recomputed. During verification, the verifier checks:

$$w' = \text{highbits}(A \cdot z - c \cdot t \mod q), \tag{6}$$

accepting the signature only if $w' = w$ and other conditions hold.

Dilithium achieves high efficiency by using the NTT for all polynomial multiplications and avoiding floating-point operations, which makes it highly suitable for FPGA implementation. However, realizing a practical FPGA accelerator for Dilithium remains non-trivial due to the need to handle intensive modular arithmetic, dynamic control flow across key, sign, and verify phases, and variable parameters across security levels.

III. Proposed Architecture

This section introduces our FPGA-based architecture for accelerating the Dilithium digital signature scheme.

A. System overview

The overall hardware architecture is shown in Fig. 1, where a central BRAM array stores all intermediate polynomial data. Functional units, including the NTT and INTT core, Keccak hashing engine, samplers, hint logic, decomposer, and signature verifiers, interact with this memory via a coordinated access scheme. Three finite state machines (FSMs) independently control the key generation, signing, and verification pipelines. These FSMs operate concurrently, ensuring non-blocking execution and maximizing throughput across all operational phases.

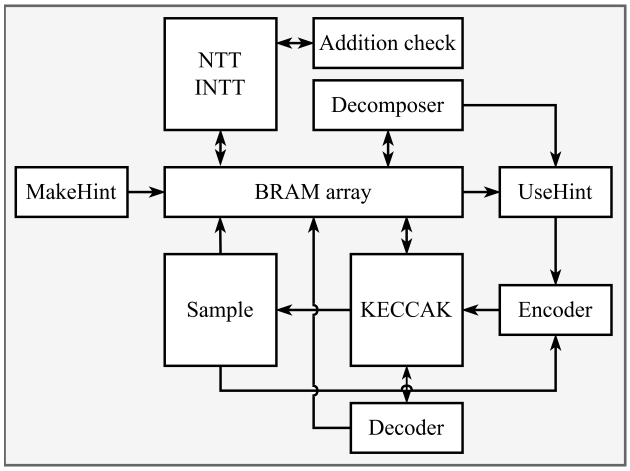

Fig. 1. Top-level hardware architecture for Dilithium.

B. 2×2 butterfly NTT engine

The core innovation in our NTT accelerator lies in the 2×2 butterfly structure, which processes two consecutive NTT stages on two input coefficient pairs per cycle. This design directly addresses inefficiencies in prior NTT implementations, where only one stage is processed per cycle and intermediate data is repeatedly written to and read from memory, introducing latency and BRAM contention.

As illustrated in Fig. 2, the engine consists of four butterfly units across two sequential stages. Each stage receives two coefficient pairs and applies the Cooley-Tukey operations [11]:

$$a' = a + wb \mod q, \text{ and } b' = a - wb \mod q \tag{7}$$

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on May 06,2026 at 19:12:28 UTC from IEEE Xplore. Restrictions apply.

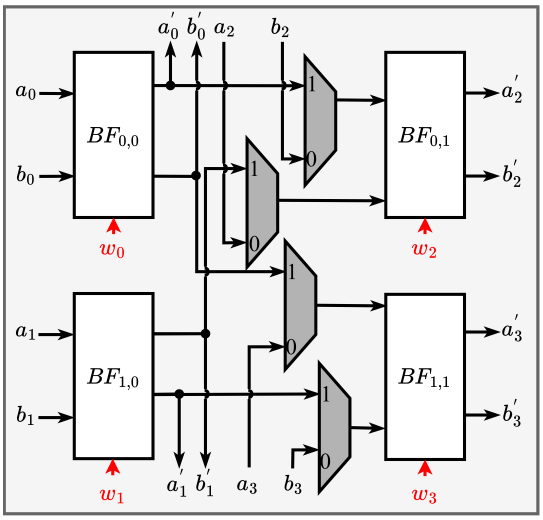

Fig. 2. (a) The routing structure of 2×2 butterfly unit and (b) Illustration of the butterfly $BF_{1,1}$ unit.

Intermediate results from stage one are forwarded internally through multiplexers, eliminating the need for intermediate memory writes. Stage two then applies additional twiddle factors w_2, w_3 to complete the transformation. This double-stage operation is supported by modular multiplication and fast Barrett reduction logic, which replaces division with bit-shifting and multiplication, reducing critical path delay.

As shown in Fig. 3, the design is fully pipelined: four coefficients are fetched each cycle, processed through two NTT stages, and then buffered for writeback. This architecture supports both forward and inverse NTTs by switching butterfly mode and twiddle read-only memory (ROM) sources. The divide-by-n in INTT is merged into the final butterfly stage using shift operations, further reducing latency. The memory system is co-optimized with the butterfly engine to enable continuous data streaming without stalls. Coefficients are arranged in a 64×4 memory format, aligned with the 4-coefficient-per-cycle processing rate of the NTT pipeline. Two dedicated 64×6 ROMs perform address resolution based on bit-reversal patterns and stage progression, ensuring proper data access sequencing.

The conflict-free design prevents simultaneous access to the same BRAM ports. This is critical for avoiding pipeline stalls, particularly when reading four coefficients and writing back the results from a double-stage NTT computation. Because

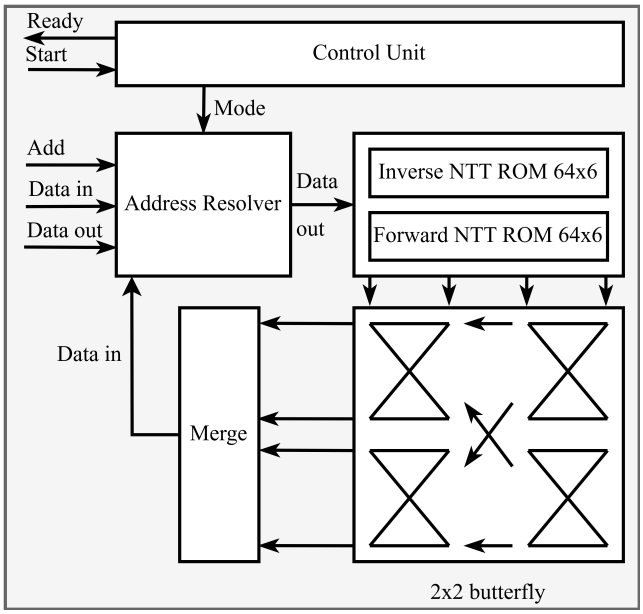

Fig. 3. High-level architecture of the NTT engine.

coefficients stay in registers across butterfly stages, the number of memory accesses is halved compared to single-stage NTT designs.

By integrating the 2×2 butterfly architecture with a memory-aware pipeline, our NTT engine achieves significant improvements in throughput and efficiency. The design computes two NTT stages per clock cycle, processing four coefficients concurrently, which greatly accelerates the polynomial transformation process. Memory contention is minimized through an address-resolved access scheme that aligns with the butterfly computation pattern, enabling uninterrupted data flow. Modular arithmetic operations are shared across both stages using Barrett-based datapaths, reducing resource overhead without compromising speed. Additionally, the architecture supports runtime reconfiguration for Dilithium security levels 2, 3, and 5, allowing a single hardware instance to adapt to varying performance and security requirements flexibly.

IV. Experimental Result

This section presents our implementation of resource utilization and performance results (Ours), comparing these outcomes with other designs. The proposed architecture is described in Verilog, with results generated for Xilinx Versal Premium Series (VPS) FPGAs, utilizing the default synthesis and implementation strategy through the Vivado tool.

To ensure a rigorous and equitable comparison of design efficiencies, we employ the area $\times$ time parameter as a key performance metric, expressed in thousands of equivalent LUTs multiplied by milliseconds (KLUTs $\times$ ms). Equivalent LUTs are calculated based on methodologies in [12], [13] as:

$$\text{LUTs}_{\text{equiv}} = \text{nLUTs} + \text{nFFs} \times 0.5 + \text{nDSPs} \times 196 \quad (8)$$

Time represents the total duration of key generation, verification, and signing. Table I offers a thorough evaluation of

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on May 06,2026 at 19:12:28 UTC from IEEE Xplore. Restrictions apply.

TABLE I
AREA-TIME PRODUCT AND EXECUTION LATENCY ACROSS SECURITY LEVELS FOR DILITHIUM HARDWARE DESIGNS

Ref.	Devices	Max freq. (MHz)	LUT (K)	FF (K)	DSP	BRAM	Keygen		Verify		Sign		Area × Time (KLUTs × ms)
							KCCs	µs	KCCs	µs	KCCs	µs	
Security level 2													
[5]	Artix-7	96.9	30	10.4	10	11	4.1	43	4.4	46	28.1/31.6	290/326	14.75
[6]	Zynq-7000	159	18.5	7.3	10	17	7.7	49	7.7	48	52	327	10.2
[9]	VUS+	270	23	9.7	4	24	14.6	54.1	15.4	57	31.7	117	6.5
[10]	Artix-7	119	30.3	13.6	16	28	3.9	33	1.9/4.2	16/36	10.7/12.8	90/108	6.36
Ours	VPS	328	55.6	30.4	8	29	5.2	15.9	6.6	20	12.7	38.87	5.4
Security level 3													
[7]	Artix-7	145	30.9	11.4	21	45	33.1	228	12.1	83	105.1	725	37.92
[5]	Artix-7	96.9	30	10.4	10	11	5.9	60	6.2	64	44.7/49.5	461/511	26.67
[6]	Zynq-7000	159	18.5	7.3	10	17	13	82	11.2	71	89.2	561	17.2
[10]	Artix-7	119	30.3	13.6	16	28	6.5	54	2.8/6.0	24/58	17.6/21/4	148/180	10.42
[9]	VUS+	270	23	9.7	4	24	23.6	87.4	26.1	96.7	48.7	180	10.4
Ours	VPS	328	55.6	30.4	8	29	9.1	27.83	9.7	29.6	19.7	59.9	8.49
Security level 5													
[8]	Artix-7	163	14	6.8	4	35	63.2	387	67.9	416	113.9	699	27.31
[6]	Zynq-7000	159	18.5	7.3	10	17	20.2	127	15.9	100	93.7	589	19.7
[9]	VUS+	270	23	9.7	4	24	39.7	147	46.7	173	70.2	260	16.6
[10]	Artix-7	119	30.3	13.6	16	28	10.9	91	4.2/11/5	35/97	20.7/27.6	174/232	14.48
Ours	VPS	328	55.6	30.4	8	29	15.8	48.1	14.6	44.6	31.2	95.32	13.61

multiple implementations of the Dilithium algorithm across different security levels, including direct comparisons with our work. The table highlights crucial metrics such as maximum frequency, area × time, resource utilization (LUTs, FFs, DSPs, and BRAM), and the number of clock cycles alongside execution times for key generation, verification, and signing processes.

Unlike prior works that rely on single-stage NTT designs, our architecture incorporates a pipelined 2×2 butterfly structure that enables simultaneous computation of two NTT stages. This drastically reduces transform latency and minimizes write-back stalls by forwarding intermediate values internally. Furthermore, we integrate Barrett reduction into each butterfly datapath, which replaces costly division operations with shift-multiply logic, boosting modular arithmetic throughput. This combination of deep pipelining, modular reuse, and conflict-free memory access forms the foundation of our performance gains. Compared to pipelined radix-4 or sparse-multiplication approaches such as [10], our design avoids hardcoded sparsity and data-specific assumptions, enabling general-purpose and scalable Dilithium acceleration.

At level 2, our design achieves a maximum frequency of 328 MHz, outperforming all others. In particular, compared to [5], [10] (119 MHz), our implementation delivers a lower area-time product of 5.4 KLUTs × ms, while maintaining superior key generation latency (15.9 µs) and competitive signing performance (38.87 µs). The efficiency gain comes from reduced memory traffic via the 2×2 butterfly pipeline and minimized overhead from Barrett-based modular arithmetic.

At level 3, we again lead in both frequency (328 MHz) and area-time efficiency. Our design achieves 8.49 KLUTs × ms, substantially outperforming [7] (37.92) and [5] (26.67). Even when compared to the optimized implementation of [10] (10.42), our pipeline achieves faster signing and key generation with fewer clock cycles, due to continuous data streaming and efficient stage-overlapping in the butterfly units.

At the highest security level, we maintain an area-time product of 13.61 KLUTs × ms, again competitive with [10] (14.48) and significantly better than [8] (27.31), despite the latter's smaller LUT footprint. Our reduced execution times (48.1 µs for key generation, 95.3 µs for signing) reflect the effectiveness of combining coefficient-level pipelining and runtime reconfigurability across polynomial lengths.

Our implementations consistently achieve higher frequencies while maintaining competitive resource efficiency across all security levels. We offer fast execution times for key generation and signing, making them suitable for applications requiring quick cryptographic operations. While our implementations use more resources than others, they achieve a good balance between speed and efficiency, especially at higher frequencies.

V. CONCLUSION

We presented a high-performance FPGA implementation of Dilithium, optimized through a 2×2 butterfly NTT architecture and conflict-free memory. Our design achieves strong performance across all security levels. Future work will explore resource optimization, scalability, and system-level integration to further advance post-quantum cryptographic hardware.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on May 06,2026 at 19:12:28 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proceedings 35th Annual Symposium on Foundations of Computer Science*, Washington, DC, US, 20-22 Nov. 1994, pp. 124–134.

[2] H. Nguyen, S. Huda, Y. Nogami, and T. T. Nguyen, "Security in post-quantum era: A comprehensive survey on lattice-based algorithms," *IEEE Access*, vol. 13, pp. 89 003–89 024, May. 2025.

[3] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Dilithium – Algorithm specifications and supporting documentation (version 3.1)," Feb. 2021, accessed 2025-01-21. [Online]. Available: https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf

[4] G. Alagic, J. Alperin-Sheriff, D. Apon, D. Cooper, J. Kelsey, Q. Dang, Y.-K. Liu, C. Miller, R. Peralta, D. Moody, R. Perlner, A. Robinson, and D. Smith-Tone, "Status report on the third round of the NIST post-quantum cryptography standardization process," National Institute of Standards and Technology, NISTIR 8413, Jul. 2022.

[5] C. Zhao, N. Zhang, H. Wang, B. Yang, W. Zhu, Z. Li, M. Zhu, S. Yin, S. Wei, and L. Liu, "A compact and high-performance hardware architecture for CRYSTALS-Dilithium," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2022, no. 1, pp. 270–295, Nov. 2021.

[6] T. Wang, C. Zhang, P. Cao, and D. Gu, "Efficient implementation of dilithium signature scheme on FPGA SoC platform," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 30, no. 9, pp. 1158–1171, Jun. 2022.

[7] G. Land, P. Sasdrich, and T. Güneysu, "A hard crystal - Implementing Dilithium on reconfigurable hardware," in *Smart Card Research and Advanced Applications*. Amsterdam, The Netherland: Springer International Publishing, 14 - 16 Nov. 2023, pp. 210–230.

[8] N. Gupta, A. Jati, A. Chattopadhyay, and G. Jha, "Lightweight hardware accelerator for post-quantum digital signature CRYSTALS-Dilithium," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 70, no. 8, pp. 3234–3243, Aug. 2023.

[9] A. Aikata, A. C. Mert, D. Jacquemin, A. Das, D. Matthews, S. Ghosh, and S. S. Roy, "A unified cryptoprocessor for lattice-based signature and key-exchange," *IEEE Transactions on Computers*, vol. 72, no. 6, pp. 1568–1580, Oct. 2023.

[10] H. Zhao, C. Zhao, W. Zhu, B. Yang, S. Wei, and L. Liu, "Sparse polynomial multiplication-based high-performance hardware implementation for CRYSTALS-Dilithium," in *2024 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, Washington DC, USA, 06 - 09 May. 2024, pp. 150–159.

[11] W. M. Gentleman and G. Sande, "Fast Fourier transforms: For fun and profit," in *Proceedings of the November 7-10, 1966, Fall Joint Computer Conference*, ser. AFIPS '66 (Fall). New York, NY, USA: Association for Computing Machinery, 07-10 Nov. 1966, pp. 563–578.

[12] Xilinx, "7 series FPGAs configurable logic block," Nov. 2014, accessed 2025-03-15. [Online]. Available: https://docs.amd.com/v/u/en-US/ug474_7Series_CLB

[13] B. Ronak and S. A. Fahmy, "Mapping for maximum performance on FPGA DSP blocks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 4, pp. 573–585, Apr. 2016.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on May 06,2026 at 19:12:28 UTC from IEEE Xplore. Restrictions apply.